

§ 7.2 二叉搜索树

7.2.1 顺序性

若二叉树中各节点所对应的词条之间支持大小比较，则在不致歧义的情况下，我们可以不必严格区分树中的节点、节点对应的词条以及词条内部所存的关键码。

如图7.2所示，在所谓的二叉搜索树（binary search tree）中，处处都满足顺序性：

任一节点 r 的左（右）子树中，所有节点（若存在）均不大于（不小于） r

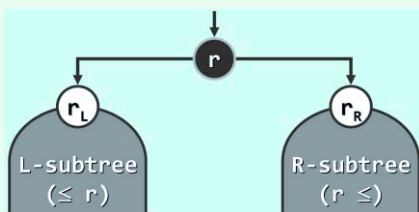


图7.2 二叉搜索树即处处满足顺序性的二叉树

为回避边界情况，这里不妨暂且假定所有节点互不相等。于是，上述顺序性便可简化表述为：

任一节点 r 的左（右）子树中，所有节点（若存在）均小于（大于） r

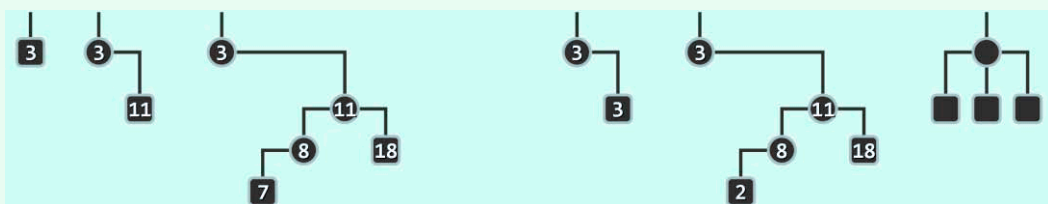


图7.3 二叉搜索树的三个实例（左），以及三个反例（右）

当然，在实际应用中，对相等元素的禁止既不自然也不必要。读者可在本书所给代码的基础上继续扩展，使得二叉搜索树的接口支持相等词条的同时并存（习题[7-10]）。比如，在去除掉这一限制之后，图7.3中原来的第一个反例，将转而成为合法的二叉搜索树。

7.2.2 中序遍历序列

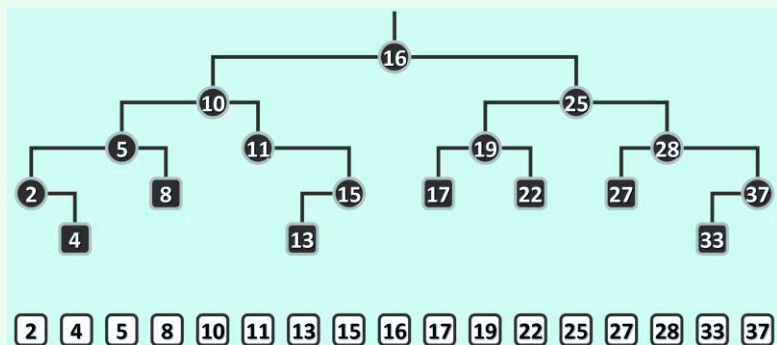


图7.4 二叉搜索树（上）的中序遍历序列（下），必然单调非降

顺序性是一项很强的条件。实际上，搜索树中节点之间的全序关系，已完全“蕴含”于这一

条件之中。以如图7.4所示的二叉搜索树为例，只需采用5.4.3节的算法对该树做一次中序遍历，即可将该树转换为一个线性序列，且该序列中的节点严格按照其大小次序排列。

这一现象，并非巧合。借助数学归纳法，可以证明更具一般性的结论（习题[7-1]）：

任何一棵二叉树是二叉搜索树，当且仅当其中序遍历序列单调非降

7.2.3 BST模板类

既然二叉搜索树属于二叉树的特例，故自然可以基于BinTree模板类（121页代码5.5），派生出如代码7.2所示的BST模板类。

```
1 #include "../BinTree/BinTree.h" //引入BinTree
2
3 template <typename T> class BST : public BinTree<T> { //由BinTree派生BST模板类
4 protected:
5     BinNodePosi(T) _hot; //BST::search()最后访问的非空（除非树空）的节点位置
6     BinNodePosi(T) connect34( //按照“3 + 4”结构，联接3个节点及四棵子树
7         BinNodePosi(T), BinNodePosi(T), BinNodePosi(T),
8         BinNodePosi(T), BinNodePosi(T), BinNodePosi(T), BinNodePosi(T));
9     BinNodePosi(T) rotateAt(BinNodePosi(T) x); //对x及其父亲、祖父做统一旋转调整
10 public: //基本接口：以virtual修饰，强制要求所有派生类（BST变种）根据各自的规则对其重写
11     virtual BinNodePosi(T) & search(const T& e); //查找
12     virtual BinNodePosi(T) insert(const T& e); //插入
13     virtual bool remove(const T& e); //删除
14 };
```

代码7.2 由BinTree派生的二叉搜索树模板类BST

可见，在继承原模板类BinTree的同时，BST内部也继续沿用了二叉树节点模板类BinNode。

按照二叉搜索树的接口规范定义，这里新增三个标准的对外接口search()、insert()和remove()，分别对应于基本的查找、插入和删除操作。这三个标准接口的调用参数，都是属于元素类型T的对象引用——这正是此类结构“循关键词访问”方式的具体体现。

另外，既然这些操作接口的语义均涉及词条的大小和相等关系，故这里也假定基本元素类型T或者直接支持比较和判等操作，或者已经重载过对应的操作符。

本章以及下一章还将以BST为基类，进一步派生出二叉搜索树的多个变种。无论哪一变种，既必须支持上述三个基本接口，同时在内部的具体实现方式又有所不同。因此，它们均被定义为虚成员函数，从而强制要求派生的所有变种，根据各自的规则对其重写。

7.2.4 查找算法及其实现

■ 算法

二叉搜索树的查找算法，亦采用了减而治之的思路与策略，其执行过程可描述为：

从树根出发，逐步地缩小查找范围，直到发现目标（成功）或缩小至空树（失败）

例如，在图7.5中查找关键词22的过程如下。

首先，经与根节点16比较确认目标关键字更大，故深入右子树25递归查找；经比较发现目标关键字更小，故继续深入左子树19递归查找；经再次比较确认目标关键字更大后，深入右子树22递归查找；最终在节点22处匹配，查找成功。

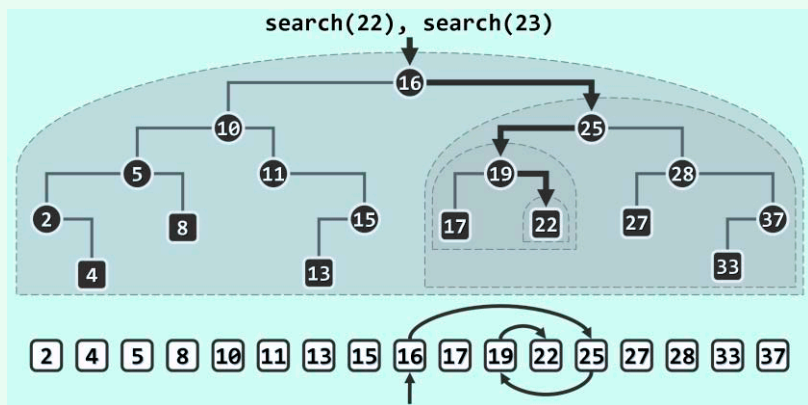


图7.5 二叉搜索树的查找过程（查找所经过的通路，以粗线条示意）

当然，查找未必成功。比如针对关键字20的查找也会经过同一查找通路并抵达节点22，但在因目标关键字更小而试图继续向左深入时发现左子树为空^①，至此即可确认查找失败。

一般地，在上述查找过程中，一旦发现当前节点为NULL，即说明查找范围已经缩小至空，查找失败；否则，视关键字比较结果，向左（更小）或向右（更大）深入，或者报告成功（相等）。

对照中序遍历序列可见，整个过程与有序向量的二分查找过程等效，故可视作后者的推广。

■ searchIn()算法与search()接口

一般地，在子树v中查找关键字e的过程，可实现为如代码7.3所示的算法searchIn()。

```
1 template <typename T> //在以v为根的 (AVL、SPLAY、rbTree等) BST子树中查找关键字e
2 static BinNodePosi(T) & searchIn (BinNodePosi(T)& v, const T& e, BinNodePosi(T)& hot) {
3     if (!v || (e == v->data)) return v; //递归基：在节点v（或假想的通配节点）处命中
4     hot = v; //一般情况：先记下当前节点，然后再
5     return searchIn(((e < v->data) ? v->lChild : v->rChild), e, hot); //深入一层，递归查找
6 } //返回时，返回值指向命中节点（或假想的通配哨兵），hot指向其父亲（退化时为初始值NULL）
```

代码7.3 二叉搜索树searchIn()算法的递归实现

节点的插入和删除操作，都需要首先调用查找算法，并根据查找结果确定后续的处理方式。因此，这里以引用方式传递（子）树根节点，以为后续操作提供必要的信息。

如代码7.4所示，通过调用searchIn()算法，即可实现二叉搜索树的标准接口search()。

```
1 template <typename T> BinNodePosi(T) & BST<T>::search(const T& e) //在BST中查找关键字e
2 { return searchIn(_root, e, _hot = NULL); } //返回目标节点位置的引用，以便后续插入、删除操作
```

代码7.4 二叉搜索树search()接口

^① 此类空节点通常对应于空孩子指针或引用，也可假想地等效为“真实”节点，后一方式不仅可简化算法描述以及退化情况的处理，也可直观地解释（B-树之类）纵贯多级存储层次的搜索树。故在后一场合，空节点也称作外部节点（external node），并等效地当作叶节点的“孩子”。这里暂采用前一方式，故空节点不在插图中出现。

■ 语义约定

以上查找算法之所以如此实现，是为了统一并简化后续不同搜索树的各种操作接口的实现。其中的技巧，主要体现于返回值和`hot`变量（即`BinTree`对象内部的`_hot`变量）的语义约定。

若查找成功，则`searchIn()`以及`search()`的返回值都将如图7.6(a)所示，指向一个关键码为`e`且真实存在的节点；若查找失败，则返回值的数值虽然为`NULL`，但是它作为引用将如图(b)所示，指向最后一次试图转向的空节点。对于后一情况，不妨假想地将此空节点转换为一个数值为`e`的哨兵节点——如此，无论成功与否，查找的返回值总是等效地指向“命中节点”。

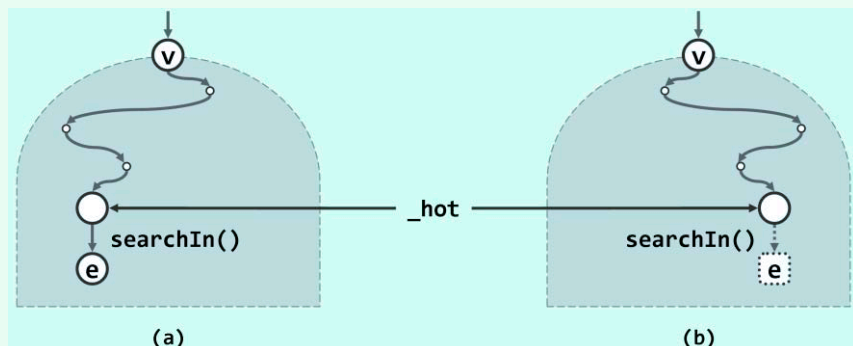


图7.6 `searchIn()`算法对返回值和`_hot`的语义定义：(a) 查找成功；(b) 查找失败

在调用`searchIn()`算法之前，`search()`接口首先将内部变量`_hot`初始化为`NULL`，然后作为引用型参数`hot`传递给`searchIn()`。在整个查找的过程中，`hot`变量始终指向当前节点的父亲。因此在算法返回时，按照如上定义，`_hot`亦将统一指向“命中节点”的父亲。

请注意，`_hot`节点是否拥有另一个孩子，与查找成功与否无关。查找成功时，节点`e`可能是叶子，也可能是内部节点；查找失败时，假想的哨兵`e`等效于叶节点，但可能有兄弟。

同时也请读者对照代码7.3验证，即便在退化的情况下（比如查找终止并返回于树根处），算法`searchIn()`的输出依然符合以上语义约定。

在7.2.6节将要介绍的删除操作中，也首先要进行查找（不妨设查找成功）。按照如上语义，命中节点必然就是待摘除节点；该节点与其父亲`_hot`，联合指示了删除操作的位置。7.2.5节将要介绍的插入操作，亦首先需做查找（不妨设查找失败）。按照如上语义，假想的“命中节点”也就是待插入的新节点；`_hot`所指向的，正是该节点可行的接入位置。

■ 效率

在二叉搜索树的每一层，查找算法至多访问一个节点，且只需常数时间，故总体所需时间应线性正比于查找路径的长度，或最终返回节点的深度。在最好情况下，目标关键码恰好出现在树根处（或其附近），此时只需 $O(1)$ 时间。然而不幸的是，对于规模为 n 的二叉搜索树，深度在最坏情况下可达 $O(n)$ 。比如，当该树退化为（接近于）一条单链时，发生此类情况的概率将很高。此时的单次查找可能需要线性时间并不奇怪，因为实际上这样的一棵“二分”搜索树，已经退化成了一个不折不扣的一维有序列表，而此时的查找则等效于顺序查找。

由此我们可得到启示：若要控制单次查找在最坏情况下的运行时间，须从控制二叉搜索树的高度入手。后续章节将要讨论的平衡二叉搜索树，正是基于这一思路而做的改进。

7.2.5 插入算法及其实现

■ 算法

为了在二叉搜索树中插入一个节点，首先需要利用查找算法`search()`确定插入的位置及方向，然后才能将新节点作为叶子插入。

以如图7.7(a)所示的二叉搜索树为例。若欲插入关键码40，则在执行`search(40)`之后，如图(b)所示，`_hot`将指向比较过的最后一个节点46，同时返回其左孩子（此时为空）的位置。于是接下来如图(c)所示，只需创建新节点40，并将其作为46的左孩子接入，拓扑意义上的节点插入即告完成。不过，为保持二叉搜索树作为数据结构的完整性和一致性，还需从节点`_hot`（46）出发，自底而上地逐个更新新节点40历代祖先的高度。

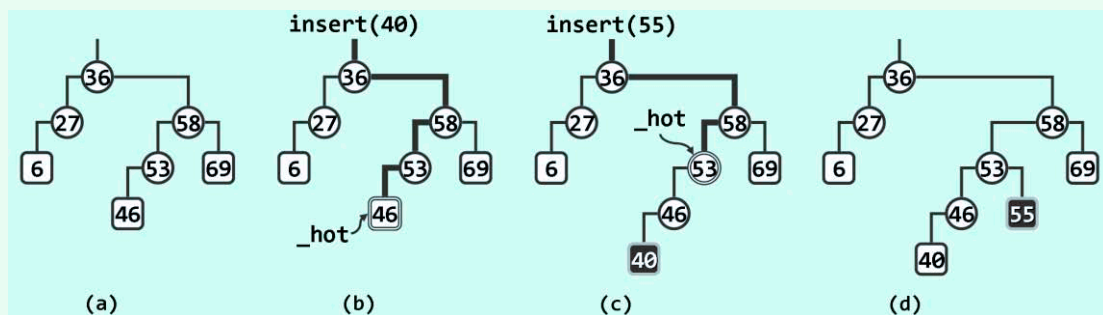


图7.7 二叉搜索树节点插入算法实例

接下来若欲插入关键码55，则在执行`search(55)`之后如图(c)所示，`_hot`将指向比较过的最后一个节点53，同时返回其右孩子（此时为空）的位置。于是如图(d)所示，创建新节点55，并将其作为53的右孩子接入。当然，此后同样需从节点`_hot`出发，逐代更新祖先的高度。

■ `insert()`接口的实现

一般地，在二叉搜索树中插入新节点`e`的过程，可描述为代码7.5中的函数`insert()`。

```
1 template <typename T> BinNodePosi(T) BST<T>::insert(const T& e) { //将关键码e插入BST树中
2     BinNodePosi(T) & x = search(e); if (x) return x; //确认目标节点不存在（留意对_hot的设置）
3     x = new BinNode<T>(e, _hot); //创建新节点x：以e为关键码，以_hot为父
4     _size++; //更新全树规模
5     updateHeightAbove(x); //更新x及其历代祖先的高度
6     return x; //新插入的节点，必为叶子
7 } //无论e是否存在于原树中，返回时总有x->data == e
```

代码7.5 二叉搜索树`insert()`接口

首先调用`search()`查找`e`。若返回位置`x`非空，则说明已有雷同节点，插入操作失败。否则，`x`必是`_hot`节点的某一空孩子，于是创建这个孩子并存入`e`。此后，更新全树的规模记录，并调用代码5.6中的`updateHeightAbove()`更新`x`及其历代祖先的高度。

注意，按照以上实现方式，无论插入操作成功与否，都会返回一个非空位置，且该处的节点与拟插入的节点相等。如此可以确保一致性，以简化后续的操作。另外，也请对照代码7.3和代码7.4中的查找算法，体会这里对“首个节点插入空树”等特殊情况的处理手法。

效率

由上可见, 节点插入操作所需的时间, 主要消耗于对算法`search()`及`updateHeightAbove()`的调用。后者与前者一样, 在每一层次至多涉及一个节点, 仅消耗 $O(1)$ 时间, 故其时间复杂度也同样取决于新节点的深度, 在最坏情况下不超过全树的高度。

7.2.6 删除算法及其实现

为从二叉搜索树中删除节点, 首先也需要调用算法`BST::search()`, 判断目标节点是否的确存在于树中。若存在, 则需返回其位置, 然后方能相应地具体实施删除操作。

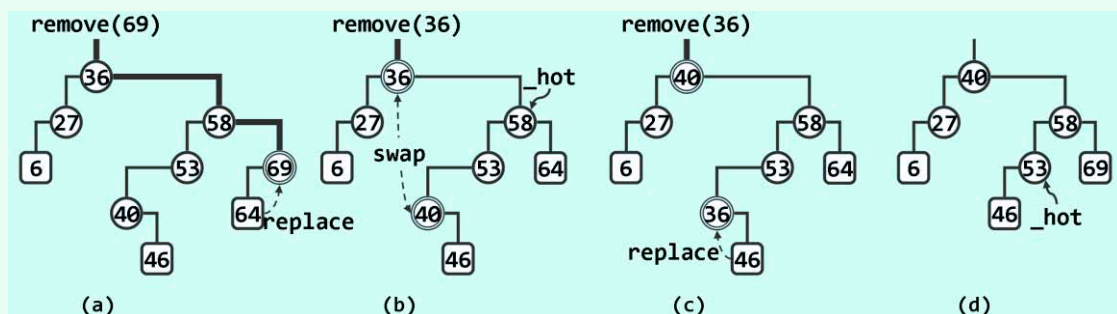


图7.8 二叉搜索树节点删除算法实例

单分支情况

以如图7.8(a)所示二叉搜索树为例。若欲删除节点69, 需首先通过`search(69)`定位待删除节点(69)。因该节点的右子树为空, 故只需如图(b)所示, 将其替换为左孩子(64), 则拓扑意义上的节点删除即告完成。当然, 为保持二叉搜索树作为数据结构的完整性和一致性, 还需更新全树的规模记录, 释放被摘除的节点(69), 并自下而上地逐个更新替代节点(64)历代祖先的高度。注意, 首个需要更新高度的祖先(58), 恰好由`_hot`指示。

不难理解, 对于没有左孩子的目标节点, 也可以对称地予以处理。当然, 以上同时也已涵盖了左、右孩子均不存在(即目标节点为叶节点)的情况。

那么, 当目标节点的左、右孩子双全时, 删除操作又该如何实施呢?

双分支情况

继续上例, 设拟再删除二度节点36。如图7.8(b)所示, 首先调用`BinNode::succ()`算法, 找到该节点的直接后继(40)。然后, 只需如图(c)所示交换二者的数据项, 则可将后继节点等效地视作待删除的目标节点。不难验证, 该后继节点必无左孩子, 从而相当于转化为此前相对简单的情况。于是最后可如图(d)所示, 将新的目标节点(36)替换为其右孩子(46)。

请注意, 在中途互换数据项之后, 这一局部如图(c)所示曾经一度并不满足顺序性。但这并不要紧——不难验证, 在按照上述方法完成整个删除操作之后, 全树的顺序性必然又将恢复。

同样地, 除了更新全树规模记录和释放被摘除节点, 此时也要更新一系列祖先节点的高度。不难验证, 此时首个需要更新高度的祖先(53), 依然恰好由`_hot`指示。

remove()

一般地, 删除关键码`e`的过程, 可描述为如代码7.6所示的函数`remove()`。

```

1 template <typename T> bool BST<T>::remove(const T& e) { //从BST树中删除关键码e
2     BinNodePosi(T) & x = search(e); if (!x) return false; //确认目标节点存在 (留意对_hot的设置)
3     removeAt(x, _hot); _size--; //实施删除
4     updateHeightAbove(_hot); //更新_hot及其历代祖先的高度
5     return true;
6 } //删除成功与否, 由返回值指示

```

代码7.6 二叉搜索树remove()接口

首先调用search()查找e。若返回位置x为空, 则说明树中不含目标节点, 故删除操作随即可以失败返回。否则, 调用removeAt()删除目标节点x。同样, 此后还需更新全树的规模, 并调用函数updateHeightAbove(_hot) (121页代码5.6), 更新被删除节点历代祖先的高度。

■ removeAt()

这里, 实质的删除操作由removeAt()负责分情况实施, 其具体实现如代码7.7所示。

```

1 /*****
2  * BST节点删除算法: 删除位置x所指的节点 (全局静态模板函数, 适用于AVL、Splay、RedBlack等各种BST)
3  * x由此前的查找确定, 并经确认非NULL后方调用本函数, 故必删除成功
4  * 与searchIn不同, 调用之前不必将hot置空
5  * 返回值指向实际被删除节点的接替者, hot指向实际被删除节点的父亲——二者均有可能是NULL
6  *****/
7 template <typename T> static BinNodePosi(T) removeAt(BinNodePosi(T) & x, BinNodePosi(T) & hot) {
8     BinNodePosi(T) w = x; //实际被摘除的节点, 初值同x
9     BinNodePosi(T) succ = NULL; //实际被删除节点的接替者
10    if (!HasLChild(*x)) //若*x的左子树为空, 则可
11        succ = x = x->rChild; //直接将*x替换为其右子树
12    else if (!HasRChild(*x)) //若右子树为空, 则可
13        succ = x = x->lChild; //对称地处理——注意: 此时succ != NULL
14    else { //若左右子树均存在, 则选择x的直接后继作为实际被摘除节点, 为此需要
15        w = w->succ(); // (在右子树中) 找到*x的直接后继*w
16        swap(x->data, w->data); //交换*x和*w的数据元素
17        BinNodePosi(T) u = w->parent;
18        ((u == x) ? u->rChild : u->lChild) = succ = w->rChild; //隔离节点*w
19    }
20    hot = w->parent; //记录实际被删除节点的父亲
21    if (succ) succ->parent = hot; //并将被删除节点的接替者与hot相联
22    release(w->data); release(w); return succ; //释放被摘除节点, 返回接替者
23 }

```

代码7.7 二叉搜索树removeAt()算法

■ 效率

删除操作所需的时间, 主要消耗于对search()、succ()和updateHeightAbove()的调用。在树中的任一高度, 它们至多消耗 $O(1)$ 时间。故总体的渐进时间复杂度, 亦不超过全树的高度。

§ 7.3 平衡二叉搜索树

7.3.1 树高与性能

根据7.2节对二叉搜索树的实现与分析，`search()`、`insert()`和`remove()`等主要接口的运行时间，均线性正比于二叉搜索树的高度。而在最坏情况下，二叉搜索树可能彻底地退化为列表，此时的查找效率甚至会降至 $O(n)$ ，线性正比于数据集的规模。因此，若不能有效地控制树高，则就实际的性能而言，较之此前的向量和列表，二叉搜索树将无法体现出明显优势。

那么，出现上述最坏（或较坏）情况的概率有多大？或者，至少从平均复杂度的角度来看，二叉搜索树的性能是否还算令人满意？

以下，将按照两种常用的随机统计口径，就二叉搜索树的平均性能做一比较。

■ 随机生成

不妨设各节点对应于 n 个互异关键码 $\{e_1, e_2, \dots, e_n\}$ 。于是按照每一排列：

$$\sigma = (e_{i_1}, e_{i_2}, \dots, e_{i_n})$$

只要从空树开始，通过依次执行`insert( $e_{i_k}$ )`，即可得到这 n 个关键码的一棵二叉搜索树 $T(\sigma)$ 。与随机排列 σ 如此相对应的二叉搜索树 $T(\sigma)$ ，称作由 σ “随机生成”（randomly generated）。

图7.9以三个关键码 $\{1, 2, 3\}$ 为例，列出了由其所有排列所生成的二叉搜索树。

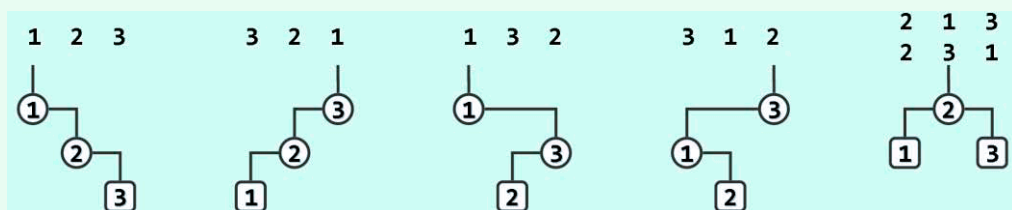


图7.9 由三个关键码 $\{1, 2, 3\}$ 的6种全排列生成的二叉搜索树

显然，任意的 n 个互异关键码，都可以构成 $n!$ 种全排列。若各排列作为输入序列的概率均等，则只要将它们各自所生成二叉搜索树的平均查找长度进行平均，即可在一定程度上反映二叉搜索树的平均查找性能。可以证明^{[29][30]}，在这一随机意义下，二叉搜索树的平均高度为 $\Theta(\log n)$ 。

■ 随机组成

另一随机策略是，假定 n 个互异节点同时给定，然后在遵守顺序性的前提下，随机确定它们之间的拓扑联接。如此，称二叉搜索树由这组节点“随机组成”（randomly composed）。

实际上，由 n 个互异节点组成的二叉搜索树，总共可能有 $(2n)!/n!/(n+1)!$ 棵（习题[7-2]）。若这些树出现的概率均等，则通过对其高度做平均可知^[30]，平均查找长度为 $\Theta(\sqrt{n})$ 。

■ 比较

前一口径的 $\Theta(\log n)$ 与后一口径的 $\Theta(\sqrt{n})$ 之间，就渐进意义而言有实质的差别。原因何在？

读者也许已经发现，同一组关键码的不同排列所生成的二叉搜索树，未必不同。仍以图7.9为例，排列 $(2, 1, 3)$ 与 $(2, 3, 1)$ 生成的，实际上就是同一棵二叉搜索树。而在按照前一口径估计平均树高时，这棵树被统计了两次。实际上一一般而言，越是平衡的树，被统计的次数亦越多。从这个角度讲，前一种平均的方式，在无形中高估了二叉搜索树的平均性能。因此相对而言，按照后一口径所得的估计值更加可信。